

JS Promises

WHAT ARE PROMISES ?



**JavaScript promises are a way
to handle asynchronous
operations in a more
structured and efficient
manner.**

**Promises allow us to write
cleaner code, avoid callback
hell, and handle errors more
gracefully.**

EXAMPLE

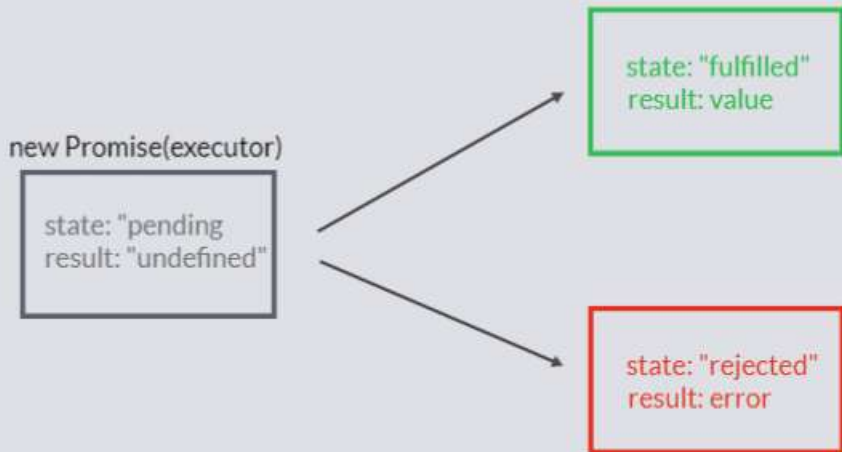


```
let promise = new Promise(function(resolve, reject) {  
  setTimeout(() => {  
    resolve('done');  
  }, 1000);  
});
```

- The function passed to new Promise is called the executor
- Its arguments resolve and reject are callbacks provided by JavaScript itself.
- Inside executor, we can perform any asynchronous operation and call resolve() or reject() depending on the result.

STATE OF PROMISE

- state — initially "pending", then changes to either "fulfilled" when resolve is called or "rejected" when reject is called.
- result — initially undefined, then changes to value when resolve(value) is called or error when reject(error) is called.



CHAINING PROMISES



```
let promise = new Promise(function(resolve, reject) {
  setTimeout(() => resolve("done!"), 1000);
});

// resolve runs the first function in .then
promise
  .then(
    result => alert(result), // shows "done!" after 1 second
  )
  .catch((error) => {
    // Handle the error from the rejected promise
  });
```

- we use the `then()` method to handle the resolved value of the previous promise, and we can use the `catch()` method to handle any errors that occur.

CLEANUP : FINALLY



```
new Promise((resolve, reject) => {  
  setTimeout(() => resolve("value"), 2000);  
})  
  .finally(() => alert("Promise ready")) // triggers first  
  .then(result => alert(result)); // ← .then shows "value"
```

- The call `.finally(f)` runs always, when the promise is settled: be it resolve or reject.
- The idea of `finally` is to set up a handler for performing cleanup/finalizing after the previous operations are complete.
- E.g. stopping loading indicators, closing no longer needed connections, etc.

PROMISE.ALL()



```
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, 'foo');
});

Promise.all([promise1, promise2, promise3]).then((values) => {
  console.log(values);
});

// Expected output: Array [3, 42, "foo"]
```

- `Promise.all()` is a method that allows us to run multiple promises in parallel and wait for all of them to complete before continuing.
- `Promise.all()` takes an array of promises as an argument and returns a new promise that resolves with an array of results.

*Thank
you*

LIKE
SHARE
FOLLOW